

Sri Lanka Institute of Information Technology

Faculty of Computing

IT2130 - Operating Systems and System Administration

Dr.Sanvitha Kasthuriarachchi

Year 02 and Semester 02

Lecture 04

Threads Management in Operating Systems

Outline

- Introduction to Threads
- Single and Multithreaded Processes
- Benefits of Threads
- User Threads and Kernel Threads
- Multithreading Models
- Thread Libraries
- Thread Pools
- Signal Handling by threads
- Thread Cancellation

Introduction to Threads

- Threads run within processes and are generally invisible from outside the processes.
- A thread represents a single, independent stream of execution within a process/ application.
- Most modern applications are multithreaded which allows a single process to have multiple threads running concurrently.
- Kernels are generally multithreaded

Introduction to Threads

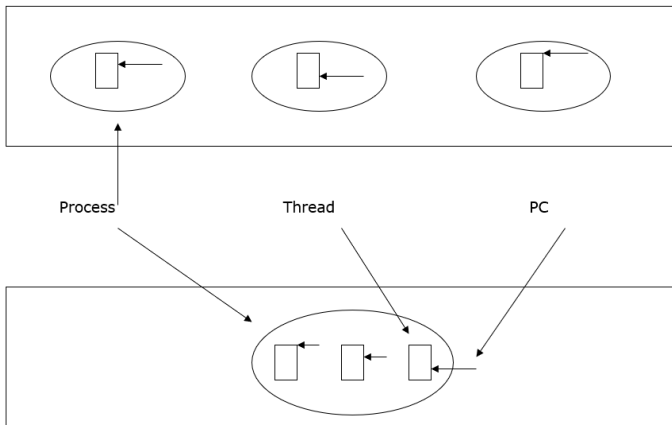
- Multiple tasks with the application can be implemented by separate threads
 - Update display
 - Fetch data
 - Spell checking
 - Answer a network request
- Process creation is heavy-weight as many resources are needed to be allocated.
- The thread creation is light-weight as there are no separate resources are needed for each threads.
- Thread is the basic unit of CPU utilization

Introduction to Threads

- Threads can be in one of several states: ready, blocked, running, or terminated.
- In a uniprocessor system, only one thread at a time is running. On a multiprocessor, different threads may execute on different processors
- Each thread has its own PC and Stack. But, all threads can access every address in the process (a thread can read or write any other thread's stack.)
- There is no protection between threads (within a process)
- Thread can create child threads, if one thread is blocked, another can run.

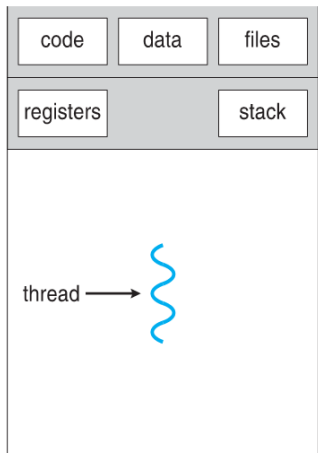
Introduction to Threads

Three processes with one thread each

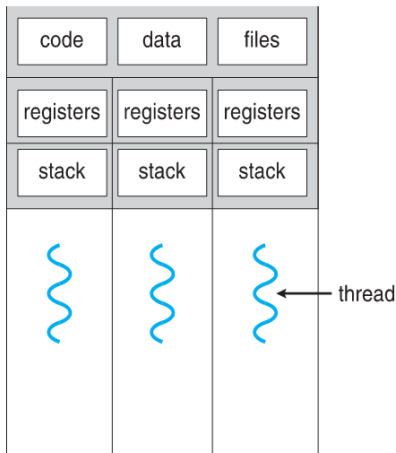


One process with three threads

Single and Multithreaded Processes



single-threaded process



multithreaded process

Benefits of Threads

- **Responsiveness** – may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing
- **Economy** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – process can take advantage of multiprocessor architectures

User Threads and Kernel Threads

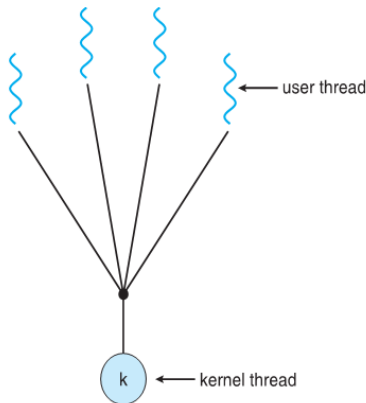
- **User threads** - management done by user-level threads library
- Three primary thread libraries:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general purpose operating systems, including:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X

Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

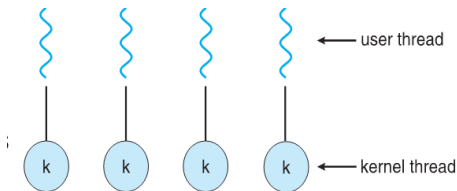
Many-to-One Model

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
 - Solaris Green Threads
 - GNU Portable Threads



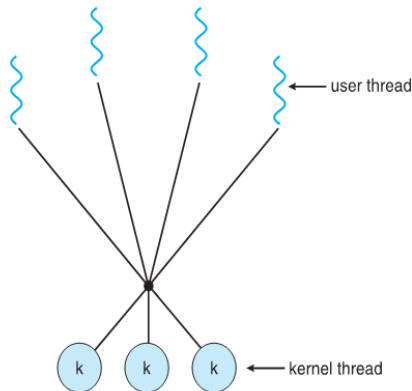
One-to-One Model

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
 - Windows
 - Linux
 - Solaris 9 and later



Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the ThreadFiber package



Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- API specifies behavior of the thread library, implementation is up to development of the library

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool
 - Separating task to be performed from mechanics of creating task allows different strategies for running task
 - i.e. Tasks could be scheduled to run periodically

Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
 - Signal is generated by particular event
 - Signal is delivered to a process
 - Signal is handled by one of two signal handlers:
 - default
 - user-defined
- Every signal has **default handler** that kernel runs when handling signal
- **User-defined signal handler** can override default
- For single-threaded, signal delivered to process

Signal Handling by Threads

Where should a signal be delivered for multi-threaded?

- Deliver the signal to the thread to which the signal applies
- Deliver the signal to every thread in the process
- Deliver the signal to certain threads in the process
- Assign a specific thread to receive all signals for the process

Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
 - **Asynchronous cancellation** terminates the target thread immediately
 - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled

Thread Example

```
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 4

void *hello(void *arg) {    /* thread main */
    printf("Hello Thread\n");
    return 0;
}

int main(void) {
    int i;
    pthread_t tid[NUM_THREADS];

    /* create/fork threads */
    for (i = 0; i < NUM_THREADS; i++) {
        pthread_create(&tid[i], NULL, hello, NULL);
    }

    for (i = 0; i < NUM_THREADS; i++) {
        pthread_join(tid[i], NULL);
    }

    return 0;
}
```

- `#include pthread.h` to create and manage POSIX threads
- `#define NUM_THREADS 4` Sets the number of threads the program will create.
- `void *hello(void *arg)` is the function executed by each thread.
- `pthread_t tid[NUM_THREADS]` Creates an array to store thread IDs.
- `pthread_create(&tid[i], NULL, hello, NULL)` Creates a new thread.
- `pthread_join(tid[i], NULL)` Ensures the main program waits for all threads to finish.
- "Hello Thread" printed four times, once from each thread

End of Lecture 4

Weekly plan

- **Practical:** Process Management using C programming
- **Workshop:**
 - Summary - Threads Management in Operating Systems
 - Discussion of Tutorial 4

